SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA		
CO1 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	M. Rohith	**Reg. No.:**	192424054
Section / Batch:	AI & DS	**Date:**	13-07-2026

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Information Extraction	Regex-Based Information Extraction	1
Pattern Matching	Regex Pattern Matching	2
Regular Expression Patterns	Regex-Based Input Validation	3

Annexure A – Application Based Questions

Section 1: Regular Expressions – Resume Information Extraction

1. A multinational recruitment company receives thousands of resumes every day in text format. The HR department requires an automated system to extract candidate information for shortlisting.

Design and implement a Python-based resume information extraction system using Regular Expressions that performs the following tasks:

Extract the candidate's name.
Identify email addresses.
Extract mobile numbers.
Detect technical skills (Python, Java, SQL, Machine Learning, NLP).
Extract years of experience.
Generate a structured summary of the candidate's profile.
Display candidates who satisfy the minimum eligibility criteria (minimum 2 years of experience and Python skill).

Section 2: Regular Expressions – Product Search System

2. An e-commerce company plans to enhance its product search engine to improve customer experience through flexible keyword matching.

Design and implement a Python-based product search system using Regular Expressions that performs the following tasks:

Search products using an exact keyword.
Perform prefix-based searches.
Perform suffix-based searches.
Search using partial keywords.
Perform case-insensitive searches.
Display all matching product names.
Generate a report showing the total number of matching products for each search.

Section 3: Regular Expressions – University Registration System

A university is developing an automated online registration portal to verify student information before course enrollment.

Design and implement a Python-based validation system using Regular Expressions that performs the following tasks:

Validate the student register number.
Validate the institutional email address.
Validate the course code.
Validate the semester information.
Validate the student's mobile number.
Display appropriate validation messages for each field.
Generate a final registration status report indicating whether the registration is successful.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Q. No. / Criteria	Understanding of Problem Context	Application of Concepts	Problem-Solving Approach	Accuracy of Solution	Clarity	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Section 1: Regular Expressions – Resume Information Extraction

Problem Context

A multinational recruitment company receives thousands of resumes in text format every day. To reduce manual effort, the HR department requires an automated system that extracts essential candidate details and identifies eligible candidates based on predefined criteria.

Learning Outcome

Develop a Python application using **Regular Expressions** to extract and validate information from resumes and generate a structured candidate summary.

Requirements

The program should:

1. Extract the candidate's name.
2. Identify valid email addresses.
3. Extract mobile numbers.
4. Detect technical skills:
    - Python
    - Java
    - SQL
    - Machine Learning
    - NLP
5. Extract years of experience.
6. Generate a structured candidate profile.
7. Display only candidates satisfying the eligibility criteria:
    - Minimum **2 years of experience**
    - Must possess **Python** skill.

Expected Output

- Candidate details displayed in a structured format.
- List of eligible candidates.

CODE:

```
import re

# Sample resumes
resumes = [
"""
Name: John Smith
Email: john.smith@gmail.com
Mobile: +91 9876543210
Skills: Python, SQL, Machine Learning, NLP
```

```
Experience: 3 years
""",

"""
Name: Priya Sharma
Email: priya123@yahoo.com
Phone: 9123456789
Skills: Java, SQL
Experience: 1 year
""",

"""
Name: David Wilson
Email: david.wilson@company.com
Contact: 9876501234
Skills: Python, Java, SQL
Experience: 5 years
"""
]

# Technical skills to search
technical_skills = ["Python", "Java", "SQL", "Machine Learning", "NLP"]

print("=" * 60)
print("RESUME INFORMATION EXTRACTION")
print("=" * 60)

eligible_candidates = []

for resume in resumes:

    # Extract Name
    name = re.search(r"Name\s*:\s*([A-Za-z ]+)", resume)
    name = name.group(1).strip() if name else "Not Found"

    # Extract Email
    email = re.search(r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}", resume)
    email = email.group() if email else "Not Found"

    # Extract Mobile Number
    mobile = re.search(r"(\+91\s?)?[6-9]\d{9}", resume)
    mobile = mobile.group() if mobile else "Not Found"

    # Extract Skills
    skills_found = []
    for skill in technical_skills:
        if re.search(skill, resume, re.IGNORECASE):
            skills_found.append(skill)

    # Extract Experience
    exp = re.search(r"(\d+)\s*year", resume, re.IGNORECASE)
    experience = int(exp.group(1)) if exp else 0
```

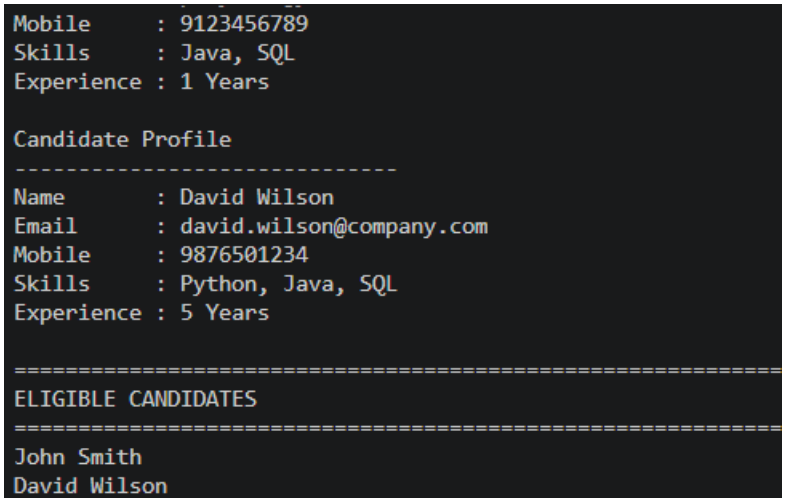
```
    # Display Summary
    print("\nCandidate Profile")
    print("-" * 30)
    print("Name       :", name)
    print("Email      :", email)
    print("Mobile     :", mobile)
    print("Skills     :", ", ".join(skills_found))
    print("Experience :", experience, "Years")

    # Eligibility Check
    if experience >= 2 and "Python" in skills_found:
        eligible_candidates.append(name)

print("\n" + "=" * 60)
print("ELIGIBLE CANDIDATES")
print("=" * 60)

if eligible_candidates:
    for candidate in eligible_candidates:
        print(candidate)
else:
    print("No eligible candidates found.")
```

OUTPUT:



Regular Expressions Used:

Purpose	**Regular Expression**
Name	Name\s*:\s*([A-Za-z]+)
Email	[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}
Mobile Number	(\+91\s?)?[6-9]\d{9}
Experience	(\d+)\s*year
Skills	re.search(skill, resume, re.IGNORECASE)

Section 2: Regular Expressions – Product Search System

Problem Context

An e-commerce company wants to improve its product search engine by allowing customers to search products using different search patterns instead of relying only on exact matches.

Learning Outcome

Develop a Python application using **Regular Expressions** to perform flexible product searches and generate search reports.

Requirements

The program should:

1. Search products using an exact keyword.
2. Perform prefix-based searches.
3. Perform suffix-based searches.
4. Search using partial keywords.
5. Perform case-insensitive searches.
6. Display all matching product names.
7. Generate a report showing the total number of matching products for each search.

Expected Output

- Matching product names for each search type.
- Number of matching products displayed after every search.

CODE:

```
import re

# Product List
products = [
    "Apple iPhone 15",
    "Apple MacBook Air",
    "Samsung Galaxy S24",
    "Samsung Smart TV",
    "Sony Headphones",
    "Dell Laptop",
    "HP Laptop",
    "Lenovo ThinkPad",
    "Wireless Mouse",
    "Bluetooth Speaker",
    "Python Programming Book",
    "Java Programming Book",
    "SQL Database Guide"
]
```

```python
# Function to perform product search
def search_products(pattern, description, flags=0):
    print("\n" + "=" * 50)
    print(description)
    print("=" * 50)

    matches = []

    for product in products:
        if re.search(pattern, product, flags):
            matches.append(product)

    if matches:
        print("Matching Products:")
        for item in matches:
            print("-", item)
    else:
        print("No matching products found.")

    print("Total Matches:", len(matches))

# 1. Exact Keyword Search
keyword = input("Enter exact keyword: ")
pattern = r"\b" + re.escape(keyword) + r"\b"
search_products(pattern, "Exact Keyword Search")

# 2. Prefix Search
prefix = input("\nEnter prefix: ")
pattern = r"^" + re.escape(prefix)
search_products(pattern, "Prefix Search")

# 3. Suffix Search
suffix = input("\nEnter suffix: ")
pattern = re.escape(suffix) + r"$"
search_products(pattern, "Suffix Search")

# 4. Partial Keyword Search
partial = input("\nEnter partial keyword: ")
pattern = re.escape(partial)
search_products(pattern, "Partial Keyword Search")

# 5. Case-Insensitive Search
case_word = input("\nEnter keyword for case-insensitive search: ")
pattern = re.escape(case_word)
search_products(pattern, "Case-Insensitive Search", re.IGNORECASE)
```

OUTPUT:

```
Enter exact keyword: Laptop

==================================================
Exact Keyword Search
==================================================
Matching Products:
- Dell Laptop
- HP Laptop
Total Matches: 2

Enter prefix: Apple

==================================================
Prefix Search
==================================================
Matching Products:
- Apple iPhone 15
- Apple MacBook Air
Total Matches: 2

Enter suffix: Book

==================================================
Suffix Search
==================================================
Matching Products:
- Python Programming Book
- Java Programming Book
Total Matches: 2

Enter partial keyword: smart

==================================================
Partial Keyword Search
==================================================
No matching products found.
Total Matches: 0

Enter keyword for case-insensitive search: samsung

==================================================
Case-Insensitive Search
==================================================
Matching Products:
- Samsung Galaxy S24
- Samsung Smart TV
Total Matches: 2
```

Regular Expressions Used:

Search Type	Regular Expression	Description
Exact Keyword	\bkeyword\b	Matches the exact word only
Prefix Search	^keyword	Matches products starting with the keyword
Suffix Search	keyword$	Matches products ending with the keyword
Partial Search	keyword	Matches the keyword anywhere in the product name
Case-Insensitive	keyword with re.IGNORECASE	Matches regardless of uppercase/lowercase

Section 3: Regular Expressions – University Registration System

Problem Context

A university is developing an online registration portal to automatically verify student information before course enrollment. The system must validate all input fields before confirming registration.

Learning Outcome

Develop a Python application using **Regular Expressions** to validate student registration details and generate a registration status report.

Requirements

The program should validate:

1. Student Register Number.
2. Institutional Email Address.
3. Course Code.
4. Semester Information.
5. Student Mobile Number.

The system should:

- Display validation messages for each field.
- Generate a final registration status report indicating whether registration is successful or failed.

Expected Output

- Validation result for each input field.
- Final registration status:
    - **Registration Successful**, or
    - **Registration Failed**

CODE:

```
import re

# Input from user
register_no = input("Enter Register Number: ")
email = input("Enter Institutional Email: ")
course_code = input("Enter Course Code: ")
semester = input("Enter Semester (1-8): ")
mobile = input("Enter Mobile Number: ")

print("\n===== VALIDATION RESULTS =====")

# 1. Validate Register Number
# Format: 22CS1012 (2 digits + 2 uppercase letters + 4 digits)
if re.fullmatch(r"\d{2}[A-Z]{2}\d{4}", register_no):
    print("Register Number : Valid")
```

```
    reg_valid = True
else:
    print("Register Number : Invalid")
    reg_valid = False

# 2. Validate Institutional Email
# Example: student@university.edu
if re.fullmatch(r"[A-Za-z0-9._%+-]+@university\.edu", email):
    print("Institutional Email : Valid")
    email_valid = True
else:
    print("Institutional Email : Invalid")
    email_valid = False

# 3. Validate Course Code
# Format: CS301, IT205, AI401
if re.fullmatch(r"[A-Z]{2,3}\d{3}", course_code):
    print("Course Code : Valid")
    course_valid = True
else:
    print("Course Code : Invalid")
    course_valid = False

# 4. Validate Semester
# Values: 1 to 8
if re.fullmatch(r"[1-8]", semester):
    print("Semester : Valid")
    sem_valid = True
else:
    print("Semester : Invalid")
    sem_valid = False

# 5. Validate Mobile Number
# Indian Mobile Number
if re.fullmatch(r"(\+91[- ]?)?[6-9]\d{9}", mobile):
    print("Mobile Number : Valid")
    mobile_valid = True
else:
    print("Mobile Number : Invalid")
    mobile_valid = False

# Final Registration Status
print("\n===== REGISTRATION STATUS =====")

if reg_valid and email_valid and course_valid and sem_valid and mobile_valid:
    print("Registration Successful")
else:
    print("Registration Failed")
```

OUTPUT:

```
Enter Register Number: 22CS1012
Enter Institutional Email: student@university.edu
Enter Course Code: CS301
Enter Semester (1-8): 5
Enter Mobile Number: 9876543210

===== VALIDATION RESULTS =====
Register Number : Valid
Institutional Email : Valid
Course Code : Valid
Semester : Valid
Mobile Number : Valid

===== REGISTRATION STATUS =====
Registration Successful
```

```
Enter Register Number: CS1012
Enter Institutional Email: student@gmail.com
Enter Course Code: C301
Enter Semester (1-8): 10
Enter Mobile Number: 1234567890

===== VALIDATION RESULTS =====
Register Number : Invalid
Institutional Email : Invalid
Course Code : Invalid
Semester : Invalid
Mobile Number : Invalid

===== REGISTRATION STATUS =====
Registration Failed
```

Regular Expressions Used:

Field	Regular Expression	Example
Register Number	\d{2}[A-Z]{2}\d{4}	22CS1012
Institutional Email	[A-Za-z0-9._%+-]+@university\.edu	student@university.edu
Course Code	[A-Z]{2,3}\d{3}	CS301, AI401
Semester	[1-8]	1 to 8
Mobile Number	(\+91[-]?)?[6-9]\d{9}	9876543210, +91 9876543210